

Exp no:

Sum of Two Sparse Polynomials using Arrays

Program

```
#include <stdio.h>

struct Polynomial
{
    int coef;
    int exp;
};

int main()
{
    struct Polynomial p1[20], p2[20], sum[40];
    int n1, n2, i = 0, j = 0, k = 0;

    printf("Enter number of terms in first polynomial: ");
    scanf("%d", &n1);

    printf("Enter coefficient and exponent:\n");
    for(i = 0; i < n1; i++)
    {
        scanf("%d %d", &p1[i].coef, &p1[i].exp);
    }

    printf("Enter number of terms in second polynomial: ");
    scanf("%d", &n2);

    printf("Enter coefficient and exponent:\n");
    for(i = 0; i < n2; i++)
    {
        scanf("%d %d", &p2[i].coef, &p2[i].exp);
    }

    i = j = k = 0;

    while(i < n1 && j < n2)
    {
        if(p1[i].exp == p2[j].exp)
        {
            sum[k].coef = p1[i].coef + p2[j].coef;
            sum[k].exp = p1[i].exp;
            i++;
            j++;
            k++;
        }
        else if(p1[i].exp > p2[j].exp)
        {
            sum[k] = p1[i];
            i++;
            k++;
        }
        else if(p2[j].exp > p1[i].exp)
        {
            sum[k] = p2[j];
            j++;
            k++;
        }
    }

    printf("Sum of two sparse polynomials is:\n");
    for(i = 0; i < k; i++)
    {
        printf("%d %d\n", sum[i].coef, sum[i].exp);
    }
}
```

```

        i++;
        k++;
    }
    else
    {
        sum[k] = p2[j];
        j++;
        k++;
    }
}

while(i < n1)
{
    sum[k] = p1[i];
    i++;
    k++;
}

while(j < n2)
{
    sum[k] = p2[j];
    j++;
    k++;
}

printf("\nResultant Polynomial:\n");

for(i = 0; i < k; i++)
{
    printf("%dx^%d", sum[i].coef, sum[i].exp);

    if(i != k - 1)
        printf(" + ");
}

printf("\n");

return 0;
}

```

Expected Output

```

Enter number of terms in first polynomial: 3
Enter coefficient and exponent:
5 3
4 2
2 0
Enter number of terms in second polynomial: 3
Enter coefficient and exponent:
3 3
2 1
1 0

Resultant Polynomial:
8x^3 + 4x^2 + 2x^1 + 3x^0

```

Exp no:

Implementation of Linear Search and Binary Search

Program

```
#include <stdio.h>

int main()
{
    int a[100], n, key;
    int i, j, temp;
    int low, high, mid;
    int lcount = 0, bcount = 0;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter the element to search: ");
    scanf("%d", &key);

    // Linear Search
    for(i = 0; i < n; i++)
    {
        lcount++;
        if(a[i] == key)
        {
            printf("\nLinear Search: Element found at position %d\n", i + 1);
            break;
        }
    }

    if(i == n)
        printf("\nLinear Search: Element not found\n");

    printf("Number of comparisons = %d\n", lcount);

    // Sort the array for Binary Search
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

```

        a[j + 1] = temp;
    }
}
}

// Binary Search
low = 0;
high = n - 1;

while(low <= high)
{
    bcount++;
    mid = (low + high) / 2;

    if(a[mid] == key)
    {
        printf("\nBinary Search: Element found.\n");
        break;
    }
    else if(a[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
}

if(low > high)
    printf("\nBinary Search: Element not found\n");

printf("Number of comparisons = %d\n", bcount);
if(lcount < bcount)
    printf("\nLinear Search performed better.\n");
else if(bcount < lcount)
    printf("\nBinary Search performed better.\n");
else
    printf("\nBoth performed equally.\n");

return 0;
}

```

Expected Output

```

Enter the number of elements: 6
Enter the elements:
40 10 60 20 50 30
Enter the element to search: 50

Linear Search: Element found at position 5
Number of comparisons = 5

Binary Search: Element found.
Number of comparisons = 2

Binary Search performed better.

```

Exp no:

Implementation of Sorting Algorithms

Aim:

Implement the following sorting algorithms:

- Insertion Sort
- Quick Sort
- Merge Sort
- Radix Sort

Compare the number of comparisons (or stages) performed by each algorithm for the same input.

Program

Insertion Sort

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[100], n;
```

```
    int i, j, key;
```

```
    int comp = 0;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for(i = 1; i < n; i++)
```

```
    {
```

```
        key = a[i];
```

```
        j = i - 1;
```

```
        while(j >= 0)
```

```
        {
```

```
            comp++;
```

```
            if(a[j] > key)
```

```
            {
```

```
                a[j + 1] = a[j];
```

```
                j--;
```

```
            }
```

```
            else
```

```
            {
```

```

        break;
    }
}

a[j + 1] = key;
}

printf("\nSorted Array:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\nNumber of Comparisons = %d\n", comp);

return 0;
}

```

Expected Output

```

Enter the number of elements: 5
Enter the elements:
45 23 78 12 56

Sorted Array:
12 23 45 56 78
Number of Comparisons = 7

```

Quick Sort

```
#include <stdio.h>

int comp = 0;

void quickSort(int a[], int low, int high);
int partition(int a[], int low, int high);

int main()
{
    int a[100], n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    printf("\nSorted Array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nNumber of Comparisons = %d", comp);

    return 0;
}

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1, j, temp;

    for(j = low; j < high; j++)
    {
        comp++;

        if(a[j] < pivot)
```

```
{  
    i++;  
  
    temp = a[i];  
    a[i] = a[j];  
    a[j] = temp;  
}  
  
temp = a[i + 1];  
a[i + 1] = a[high];  
a[high] = temp;  
  
return i + 1;  
}
```

Expected Output

```
Enter the number of elements: 5  
Enter the elements:  
45 23 78 12 56  
  
Sorted Array:  
12 23 45 56 78  
Number of Comparisons = 7
```


Merge Sort

```
#include <stdio.h>

int comp = 0;

void mergeSort(int a[], int low, int high);
void merge(int a[], int low, int mid, int high);

int main()
{
    int a[100], n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("\nSorted Array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nNumber of Comparisons = %d\n", comp);

    return 0;
}

void mergeSort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

void merge(int a[], int low, int mid, int high)
{
    int temp[100];
    int i = low;
    int j = mid + 1;
    int k = low;
```

```
while(i <= mid && j <= high)
{
    comp++;

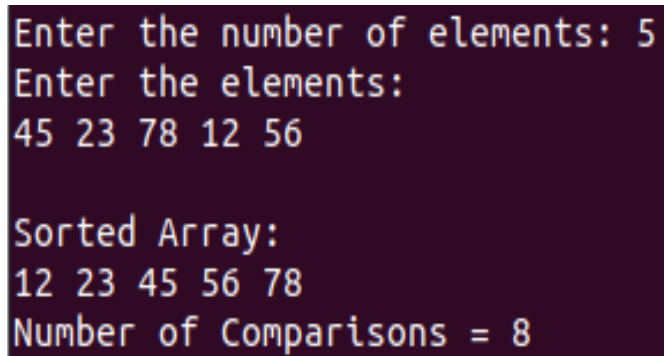
    if(a[i] <= a[j])
        temp[k++] = a[i++];
    else
        temp[k++] = a[j++];
}

while(i <= mid)
    temp[k++] = a[i++];

while(j <= high)
    temp[k++] = a[j++];

for(i = low; i <= high; i++)
    a[i] = temp[i];
}
```

Expected Output

A screenshot of a terminal window with a dark purple background and light blue text. The text shows the input and output of a sorting program. The input consists of 5 elements: 45, 23, 78, 12, and 56. The output shows the sorted array as 12, 23, 45, 56, and 78, and the total number of comparisons as 8.

```
Enter the number of elements: 5
Enter the elements:
45 23 78 12 56

Sorted Array:
12 23 45 56 78
Number of Comparisons = 8
```

Radix Sort

```
#include <stdio.h>

int stage = 0;

int getMax(int a[], int n)
{
    int i, max = a[0];

    for(i = 1; i < n; i++)
    {
        if(a[i] > max)
            max = a[i];
    }

    return max;
}

void countSort(int a[], int n, int exp)
{
    int output[100];
    int count[10] = {0};
    int i;

    for(i = 0; i < n; i++)
        count[(a[i] / exp) % 10]++;

    for(i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for(i = n - 1; i >= 0; i--)
    {
        output[count[(a[i] / exp) % 10] - 1] = a[i];
        count[(a[i] / exp) % 10]--;
    }

    for(i = 0; i < n; i++)
        a[i] = output[i];

    stage++;
}

void radixSort(int a[], int n)
{
    int exp;
    int max = getMax(a, n);

    for(exp = 1; max / exp > 0; exp *= 10)
```

```

        countSort(a, n, exp);
    }

int main()
{
    int a[100], n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    radixSort(a, n);

    printf("\nSorted Array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nNumber of Stages = %d\n", stage);

    return 0;
}

```

Expected Output

```

Enter the number of elements: 5
Enter the elements:
45 23 78 12 56

Sorted Array:
12 23 45 56 78
Number of Stages = 2

```

Exp no:

Implementing Stack using Arrays

Pre-Lab Assignment: Stack Using Arrays

1. What is a stack? Explain the LIFO principle with a suitable example.
2. How is a stack represented using an array? Explain the purpose of the TOP variable.
3. What is the initial value of TOP in an empty stack? Why is this value used?
4. Write the conditions used to determine whether:
 - (a) the stack is empty;
 - (b) the stack is full.
5. What is stack overflow? During which operation can it occur?
6. What is stack underflow? During which operation can it occur?
7. What is the difference between the Pop and Peek operations?
8. How are all stack elements displayed from top to bottom?
9. Consider a stack of size 5. Perform the following operations and show the value of TOP after each operation:
 - (a) Push(10)
 - (b) Push(20)
 - (c) Push(30)
 - (d) Pop()
 - (e) Push(40)
 - (f) Peek()
10. Consider the following stack:
TOP -> 50 40 30 20 10
Answer the following questions:
 - (a) What element is returned by Peek?
 - (b) What element is removed by Pop?
 - (c) What is the new top element after the Pop operation?

Aim:

Implement a stack using arrays and perform the operations Push,Pop,Peek,Display and Exit.

Program

```
#include <stdio.h>

int main()
{
    int size, top = -1;
    int choice, value, i;

    printf("Enter the size of the stack: ");
    scanf("%d", &size);

    int stack[size];

    do
    {
        printf("\n----- STACK MENU ----- \n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                if (top == size - 1)
                {
                    printf("Stack Overflow\n");
                }
                else
                {
                    printf("Enter the element: ");
                    scanf("%d", &value);
                    top++;
                    stack[top] = value;
                    printf("%d pushed into stack.\n", value);
                }
                break;

            case 2:
```

```

        if (top == -1)
        {
            printf("Stack Underflow\n");
        }
        else
        {
            printf("Deleted element: %d\n", stack[top]);
            top--;
        }
        break;

    case 3:
        if (top == -1)
        {
            printf("Stack is Empty\n");
        }
        else
        {
            printf("Top element: %d\n", stack[top]);
        }
        break;

    case 4:
        if (top == -1)
        {
            printf("Stack is Empty\n");
        }
        else
        {
            printf("Stack elements are:\n");
            for (i = top; i >= 0; i--)
            {
                printf("%d\n", stack[i]);
            }
        }
        break;

    case 5:
        printf("Exiting...\n");
        break;

    default:
        printf("Invalid Choice\n");
}

} while (choice != 5);

return 0;
}

```

Expected Output

```
Enter the size of the stack: 4
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 1
```

```
Enter the element: 5
```

```
5 pushed into stack.
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 1
```

```
Enter the element: 7
```

```
7 pushed into stack.
```


Exp no:

Implementation of Simple Queue Using an Array

Aim:

To implement a simple queue using a one- dimensional arrays and perform the operations Push,Pop,Peek,Display and Exit.

Pre-Lab Assignment: Stack Using Arrays

1. What is a queue? Explain the FIFO principle with a suitable example.
2. How is a queue represented using an array?
3. Explain the purpose of the front and rear variables.
4. Write the condition used to determine whether a simple queue is
 - (a) full
 - (b) empty
5. What is queue overflow and queue underflow? During which operations can it occur?
6. What is the difference between the Dequeue and Peek operations?
7. Explain the limitation of implementing a simple queue using an array.
8. Consider a simple queue of size 5. Initially:
front = -1, rear = -1

Perform the following operations and write the value of front and rear after each operation:

- (a) Enqueue(10)
 - (b) Enqueue(20)
 - (c) Enqueue(30)
 - (d) Dequeue()
 - (e) Enqueue(40)
 - (f) Enqueue(50)
9. Consider the following queue:
front → 15, 25, 35, 45 ← rear
- Answer the following questions:
- (a) Which element is returned by Peek?
 - (b) Which element is removed by Dequeue?
 - (c) What is the new front element after Dequeue?
 - (d) Which element is stored at the rear?
10. Predict the result of the following sequence of operations:
Enqueue(5), Enqueue(10), Enqueue(15), Dequeue(), P eek()

Program

```
#include <stdio.h>

int main()
{
    int queue[100];
    int size, front = -1, rear = -1;
    int choice, value, i;

    printf("Enter the size of the queue: ");
    scanf("%d", &size);

    while (1)
    {
        printf("\n--- QUEUE MENU ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                /* Enqueue */
                if (rear == size - 1)
                {
                    printf("Queue Overflow\n");
                }
                else
                {
                    printf("Enter the element: ");
                    scanf("%d", &value);

                    if (front == -1)
                        front = 0;

                    rear++;
                    queue[rear] = value;

                    printf("%d inserted into the queue\n", value);
                }
                break;

            case 2:
                /* Dequeue */
```

```

if (front == -1 || front > rear)
{
    printf("Queue Underflow\n");
}
else
{
    printf("%d deleted from the queue\n", queue[front]);
    front++;

    if (front > rear)
    {
        front = -1;
        rear = -1;
    }
}
break;

case 3:
    /* Peek */
    if (front == -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Front element = %d\n", queue[front]);
    }
    break;

case 4:
    /* Display */
    if (front == -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue elements are: ");

        for (i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }

        printf("\n");
    }
    break;

case 5:
    printf("Exiting...\n");
    return 0;

```

```
        default:
            printf("Invalid choice\n");
    }
}

return 0;
}
```

Expected Output

```
Enter the size of the queue: 3

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 23
23 inserted into the queue

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 43
43 inserted into the queue
```

Exp no:

Implementation of Circular Queue Using an Array

Aim:

To implement a circular queue using a one-dimensional array and perform the following operations:

1. Enqueue
2. Dequeue
3. Peek
4. Display

Pre-Lab Assignment

1. What is a circular queue?
2. How is a circular queue different from a simple queue?
3. Explain the purpose of the front and rear variables in a circular queue.
4. What are the initial values of front and rear in an empty circular queue?
5. Consider a circular queue of size 5. Initially:

front = -1, rear = -1

Perform the following operations:

- (a) Enqueue(10)
- (b) Enqueue(20)
- (c) Enqueue(30)
- (d) Enqueue(40)
- (e) Dequeue()
- (f) Dequeue()
- (g) Enqueue(50)
- (h) Enqueue(60)

After each operation, show:

- The contents of the array
- The logical order of queue elements
- The value of front
- The value of rear

6. Predict the result of the following operations:

Enqueue(5), Enqueue(10), Enqueue(15), Dequeue(), Enqueue(20), Peek()

Program

```
#include <stdio.h>

int main()
{
    int queue[100];
    int max, front = -1, rear = -1;
    int choice, item, i;

    printf("Enter the size of circular queue: ");
    scanf("%d", &max);

    while (1)
    {
        printf("\n----- CIRCULAR QUEUE MENU ----- \n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                /* Enqueue */
                if ((rear + 1) % max == front)
                {
                    printf("Circular Queue Overflow\n");
                }
                else
                {
                    printf("Enter the element: ");
                    scanf("%d", &item);

                    if (front == -1)
                    {
                        front = 0;
                        rear = 0;
                    }
                    else
                    {
                        rear = (rear + 1) % max;
                    }

                    queue[rear] = item;
                    printf("Element inserted successfully\n");
                }
                break;
```

case 2:

```
/* Dequeue */
if (front == -1)
{
    printf("Circular Queue Underflow\n");
}
else
{
    item = queue[front];
    printf("Deleted element: %d\n", item);

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % max;
    }
}
break;
```

case 3:

```
/* Peek */
if (front == -1)
{
    printf("Circular Queue is Empty\n");
}
else
{
    printf("Front element: %d\n", queue[front]);
}
break;
```

case 4:

```
/* Display */
if (front == -1)
{
    printf("Circular Queue is Empty\n");
}
else
{
    printf("Elements in the circular queue: ");

    i = front;

    while (1)
    {
        printf("%d ", queue[i]);
```

```

        if (i == rear)
        {
            break;
        }

        i = (i + 1) % max;
    }

    printf("\n");
}
break;

case 5:
    printf("Program terminated.\n");
    return 0;

default:
    printf("Invalid Choice\n");
}
}

return 0;
}

```

Expected Output

```

Enter the size of circular queue: 5

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 10
Element inserted successfully

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue

```


Exp no:

Implementation of Double Ended Queue (Deque) Using an Array

Aim:

To implement a Double Ended Queue (Deque) using a one-dimensional array and perform the following operations:

1. Insert an element at the front.
2. Insert an element at the rear.
3. Delete an element from the front.
4. Delete an element from the rear.
5. Display the elements of the deque.

Pre-Lab Assignment

1. What is a Double Ended Queue?
2. Why is a Double Ended Queue called a deque?
3. How is a deque different from a simple queue?
4. List the four basic operations supported by a deque.
5. Consider a deque of size 5. Initially:

front = -1, rear = -1

Perform the following operations:

- (a) InsertRear(20)
- (b) InsertRear(30)
- (c) InsertFront(10)
- (d) DeleteRear()
- (e) InsertFront(5)
- (f) DeleteFront()

After each operation, show:

- The logical contents of the deque.
- The contents of the array.
- The value of front.
- The value of rear.

6. Consider the following deque:

front → 10, 20, 30, 40 ← rear

Answer the following questions:

- (a) Which element is deleted by DeleteFront?
- (b) Which element is deleted by DeleteRear?
- (c) Which element is displayed as the front element?

(d) Which element is displayed as the rear element?

Program

```
#include <stdio.h>

int main()
{
    int max, deque[100];
    int front = -1, rear = -1;
    int choice, item, i;

    printf("Enter the size of deque: ");
    scanf("%d", &max);

    while (1)
    {
        printf("\n----- DEQUE MENU ----- \n");
        printf("1. Insert at Front\n");
        printf("2. Insert at Rear\n");
        printf("3. Delete from Front\n");
        printf("4. Delete from Rear\n");
        printf("5. Display Front Element\n");
        printf("6. Display Rear Element\n");
        printf("7. Display All Elements\n");
        printf("8. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                /* Insert at Front */
                if ((front == 0 && rear == max - 1) ||
                    (front == rear + 1))
                {
                    printf("Deque Overflow\n");
                }
                else
                {
                    printf("Enter the element: ");
                    scanf("%d", &item);

                    if (front == -1)
                    {
                        front = 0;
                        rear = 0;
                    }
                    else if (front == 0)
                    {
                        front = max - 1;
                    }
                }
            }
        }
    }
```

```

else
{
    front = front - 1;
}

deque[front] = item;
printf("Element inserted at the front.\n");
}
break;

```

case 2:

```

/* Insert at Rear */
if ((front == 0 && rear == max - 1) ||
    (front == rear + 1))
{
    printf("Deque Overflow\n");
}
else
{
    printf("Enter the element: ");
    scanf("%d", &item);

    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    else if (rear == max - 1)
    {
        rear = 0;
    }
    else
    {
        rear = rear + 1;
    }

    deque[rear] = item;
    printf("Element inserted at the rear.\n");
}
break;

```

case 3:

```

/* Delete from Front */
if (front == -1)
{
    printf("Deque Underflow\n");
}
else
{
    item = deque[front];
    printf("Deleted element: %d\n", item);
}

```

```

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else if (front == max - 1)
    {
        front = 0;
    }
    else
    {
        front = front + 1;
    }
}
break;

```

case 4:

```

/* Delete from Rear */
if (front == -1)
{
    printf("Deque Underflow\n");
}
else
{
    item = deque[rear];
    printf("Deleted element: %d\n", item);

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else if (rear == 0)
    {
        rear = max - 1;
    }
    else
    {
        rear = rear - 1;
    }
}
break;

```

case 5:

```

/* Display Front Element */
if (front == -1)
{
    printf("Deque is Empty\n");
}
else
{
    printf("Front element: %d\n", deque[front]);
}

```

```
}  
break;
```

case 6:

```
/* Display Rear Element */  
if (front == -1)  
{  
    printf("Deque is Empty\n");  
}  
else  
{  
    printf("Rear element: %d\n", deque[rear]);  
}  
break;
```

case 7:

```
/* Display All Elements */  
if (front == -1)  
{  
    printf("Deque is Empty\n");  
}  
else  
{  
    printf("Elements in the deque: ");  
  
    i = front;  
  
    while (1)  
    {  
        printf("%d ", deque[i]);  
  
        if (i == rear)  
            break;  
  
        i = (i + 1) % max;  
    }  
  
    printf("\n");  
}  
break;
```

case 8:

```
printf("Program terminated.\n");  
return 0;
```

default:

```
printf("Invalid Choice\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

Expected Output

```
Enter the size of deque: 5

----- DEQUE MENU -----
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display Front Element
6. Display Rear Element
7. Display All Elements
8. Exit
Enter your choice: 2
Enter the element: 10
Element inserted at the rear.

----- DEQUE MENU -----
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display Front Element
6. Display Rear Element
7. Display All Elements
8. Exit
```